

Evo Aura 41 Python Files - New Alteration and Flow Guide

This document defines the target 41-file architecture, each file's duty, source mapping from the existing uploaded files, app boot flow, route flow, database flow, and EXE flow. Use this as the new Codex implementation blueprint.

Final platform: PyQt6, SQLite, Python 3.11/3.12, Windows 10/11. Do not change code for Windows 7/8.

1. Final 41 Python file list

No.	Final file	Description	Source
1	main.py	Application entry point only: create QApplication, apply style, open MainWindow.	evoaura_app(6).py
2	core/app_shell.py	MainWindow/AppShell, sidebar container, QStackedWidget page cache, logout, navigation.	evoaura_app(6).py
3	core/sidebar.py	Apple-style PyQt6 collapsible sidebar and NAV_ITEMS.	sidebar(2).py
4	core/routes.py	Dictionary mapping route keys to page classes/builders and page titles.	sidebar(2).py + evoaura_app(6).py
5	core/theme.py	Central C colors, APP_SS, button styles, table styles, field styles.	evoaura_app(6).py + product_page(7).py
6	core/ui_helpers.py	Reusable QLabel/button/card/table helpers, Toast, SI, GB, combo delegate.	evoaura_app(6).py + product_page(7).py
7	core/db.py	SQLite connection, AppData database folder, company/user base methods.	evoaura_app(6).py
8	core/migrations.py	All CREATE TABLE/ALTER TABLE logic and indexes for every module.	all init_* functions
9	core/auth_service.py	Password hashing, TOTP, master auth, recovery code logic.	evoaura_app(6).py
10	core/input_behavior.py	Global input guard for spinbox/combobox wheel behavior.	input_behavior.py
11	core/app_branding.py	Window/app icon loading from assets/icons/logo.png.	app_branding.py
12	pages/auth/ask_db_page.py	Ask DB name/workspace creation page.	V_AskDB
13	pages/auth/master_auth_page.py	Master TOTP verification page.	V_MasterAuth
14	pages/auth/company_setup_page.py	Company logo/name/phone/address/GST/footer setup page.	V_CompanySettings
15	pages/auth/signup_page.py	Admin/user signup page.	V_Signup
16	pages/auth/qr_setup_page.py	Authenticator QR and recovery codes page.	V_QR
17	pages/auth/login_page.py	Username/password login page.	V_Login
18	pages/auth/otp_page.py	OTP/recovery-code verification page.	V_OTP
19	pages/dashboard_page.py	Home dashboard with KPIs, recent invoices, low-stock, quick actions.	_DashboardPage + dashboard(1).py converted
20	pages/sales/new_sale_page.py	POS billing/new sale page. Create real page; billing_page.py currently missing.	new
21	pages/sales/returns_page.py	Sales return/credit note page.	new
22	pages/sales/bill_view_page.py	Invoice list/view/print/reprint page.	new
23	pages/inventory/product_page.py	ProductPage controller layout only.	product_page(7).py

No.	Final file	Description	Source
24	pages/inventory/purchase_invoices_page.py	Purchase invoice/stock-in page.	purchase_orders_page.py
25	pages/inventory/supplier_page.py	Supplier master page.	supplier_page(1).py
26	pages/inventory/reorder_control_page.py	Low stock/reorder control page.	low_stock_page.py
27	pages/customers/customer_center_page.py	Customer master/ledger/history page.	customer_page.py
28	pages/customers/due_collection_page.py	Credit and due collection page.	credit_management_page.py
29	pages/customers/loyalty_page.py	Loyalty points/settings page.	loyalty_page.py
30	pages/finance/pl_summary_page.py	Profit and loss summary page.	new
31	pages/finance/expense_tracking_page.py	Expense entry and category tracking page.	new
32	pages/finance/gst_breakdown_page.py	GST summary by date/invoice/category.	new
33	pages/finance/cashflow_page.py	Cash in/out flow timeline.	new
34	pages/reports/daybook_page.py	Daily business summary page.	new
35	pages/reports/stock_report_page.py	Stock valuation and movement report.	new
36	pages/reports/trial_balance_page.py	Trial balance/accounting summary placeholder.	new
37	pages/reports/export_page.py	Export CSV/PDF/Excel reports page.	new
38	pages/settings/company_settings_page.py	Edit company settings after login.	V_CompanySettings reused
39	pages/settings/users_roles_page.py	Users, roles, permissions, reset recovery codes.	evoaura_app(6).py DB users
40	pages/settings/security_page.py	Password/TOTP/security settings.	auth_service + users table
41	pages/settings/audit_log_page.py	Central audit log viewer.	product/supplier/customer audit sources

2. Folder structure

```
evo_aura/  
main.py  
core/  
app_shell.py  
sidebar.py  
routes.py  
theme.py  
ui_helpers.py  
db.py  
migrations.py  
auth_service.py  
input_behavior.py  
app_branding.py  
pages/  
auth/  
sales/  
inventory/  
customers/  
finance/  
reports/  
settings/  
services/  
repositories/  
models/  
assets/icons/  
data/
```

3. Boot and authentication flow

Stage	Flow
Start	main.py creates QApplication, applies core/theme.py APP_SS, installs input guard, opens MainWindow.
Locate DB folder	core/db.py resolves %LOCALAPPDATA%/EvoAura/databases on Windows 10/11.
No DB	Ask DB -> Master Auth -> Create DB -> Run migrations -> Company Setup -> Admin Signup -> QR -> Login.
DB exists but no users	Signup -> QR -> Login.
DB exists and users exist	Login -> OTP -> AppShell/Dashboard.
Logout	Close shell, clear user state, return to login.

4. Sidebar route flow

The route keys must remain consistent with the existing sidebar NAV_ITEMS. AppShell must lazy-load each page and cache it after first use.

Route key	File to load	Page title
home	pages/dashboard_page.py	Dashboard
sale	pages/sales/new_sale_page.py	New Sale
returns	pages/sales/returns_page.py	Returns
bill_view	pages/sales/bill_view_page.py	Bill View
products	pages/inventory/product_page.py	Products
purchase_orders	pages/inventory/purchase_invoices_page.py	Purchase Invoices
suppliers	pages/inventory/supplier_page.py	Suppliers
low_stock	pages/inventory/reorder_control_page.py	Reorder Control
customers	pages/customers/customer_center_page.py	Customer Center

Route key	File to load	Page title
credit	pages/customers/due_collection_page.py	Due Collection
loyalty	pages/customers/loyalty_page.py	Loyalty
pl_summary	pages/finance/pl_summary_page.py	P & L Summary
expenses	pages/finance/expense_tracking_page.py	Expense Tracking
gst	pages/finance/gst_breakdown_page.py	GST Breakdown
cashflow	pages/finance/cashflow_page.py	Cash Flow
day_book	pages/reports/daybook_page.py	Day Book
stock_report	pages/reports/stock_report_page.py	Stock Report
trial_balance	pages/reports/trial_balance_page.py	Trial Balance
export	pages/reports/export_page.py	Export
company_settings	pages/settings/company_settings_page.py	Company
users	pages/settings/users_roles_page.py	Users & Roles
security	pages/settings/security_page.py	Security
audit_log	pages/settings/audit_log_page.py	Audit Log

5. AppShell route loading pattern

```
# core/app_shell.py - target pattern
class AppShell(QMainWindow):
    def show_page(self, key: str):
        key = canonical_key(key)
        if key not in self.page_cache:
            builder = ROUTES[key].builder
            self.page_cache[key] = builder(
                db_name=self.db_name,
                company_name=self.company_name,
                current_user=self.current_user,
                on_back=lambda: self.show_page("home"),
            )
        self.stack.addWidget(self.page_cache[key])
        self.stack.setCurrentWidget(self.page_cache[key])
        self.sidebar.set_active(key)
        self.topbar.set_title(ROUTES[key].title)
```

6. Data and code responsibility flow

Layer	What belongs here	Example
Page	UI layout, signals, table view, forms. No raw complex SQL.	pages/inventory/product_page.py
Service	Business rules, validation, calculations, workflow orchestration.	services/product_service.py
Repository	SQLite queries only. Returns dict/list values.	repositories/product_repository.py
Migration	CREATE TABLE, ALTER TABLE, indexes.	core/migrations.py
Model	Dataclass/default values/field normalization.	models/product.py

7. Suggested support files beyond the 41 UI/core files

The requested 41 files are the main UI/core pages. For clean code, Codex may also create service/repository/model files. These are not extra pages; they are backend support modules. Recommended support modules:

- services/product_service.py, sales_service.py, customer_service.py, supplier_service.py, purchase_service.py, loyalty_service.py, report_service.py
- repositories/product_repository.py, sales_repository.py, customer_repository.py, supplier_repository.py, purchase_repository.py, loyalty_repository.py
- models/product.py, customer.py, supplier.py, sale.py, purchase.py

8. Database and migration flow

- core/db.py must not create all business tables directly. It should open the connection and call run_migrations(db_path).
- core/migrations.py should contain versioned migrations and create all tables from existing init_product_table, init_supplier_tables, init_purchase_tables, init_customer_tables, init_credit_tables, init_loyalty_tables, and init_low_stock_tables.
- Create indexes for fast lookup: products(item_code), products(barcode), products(sku), products(name), sales(invoice_no), sales(invoice_date), customers(phone), suppliers(code), suppliers(name).
- Every new computer creates its own DB under AppData. Do not place company DB in the installed EXE folder.

9. File creation order for Codex

Phase 1 - Core foundation

Create core/theme.py, core/ui_helpers.py, core/db.py, core/migrations.py, core/auth_service.py, core/input_behavior.py, core/app_branding.py.

Phase 2 - Auth pages

Move V_AskDB, V_MasterAuth, V_CompanySettings, V_Signup, V_QR, V_Login, V_OTP into pages/auth files.

Phase 3 - Shell and routes

Create core/sidebar.py, core/routes.py, core/app_shell.py, main.py. App must start and login before continuing.

Phase 4 - Existing pages

Move product, supplier, purchase invoices, reorder control, customer center, due collection, and loyalty pages into pages folders.

Phase 5 - Split product and supplier internals

Move SQL to repositories and logic to services. Keep page files focused on UI.

Phase 6 - Missing pages

Create sales, finance, reports, settings pages. Start with simple working pages, then add full features.

Phase 7 - Test and package

Run py_compile, open every route, then build EXE with PyInstaller.

10. Root import cleanup

After migration, update imports to use target modules. Examples:

```
# Before
from product_page import C, FIELD_SS, _NO_ARROW, _apply_combo_delegate, init_product_table
from supplier_page import init_supplier_tables
from sidebar import Sidebar

# After
from core.theme import C, FIELD_SS
from core.ui_helpers import _NO_ARROW, _apply_combo_delegate
from core.migrations import init_product_table, init_supplier_tables
from core.sidebar import Sidebar
```

11. EXE build flow for Windows 10/11

Step	Action
1	Create and test virtual environment with Python 3.11 or 3.12.
2	Install dependencies: PyQt6, pyotp, qrcode[pil], Pillow, pyinstaller.
3	Run python main.py and test the complete DB creation/login flow.
4	Confirm all data writes to %LOCALAPPDATA%/EvoAura/databases.
5	Build EXE with PyInstaller --windowed --name EvoAura main.py and include assets/icons.
6	Test clean install on Windows 10/11 user account without admin database permissions.
7	After first run, confirm new company DB creation works from inside the installed EXE.

```
Example command:
pyinstaller --noconfirm --windowed --name EvoAura ^
--add-data "assets;assets" ^
--add-data "icons;icons" ^
main.py
```

12. Final test matrix

- Import test: python -m py_compile main.py core/*.py pages/auth/*.py pages/sales/*.py pages/inventory/*.py pages/customers/*.py pages/finance/*.py pages/reports/*.py pages/settings/*.py
- Boot test: delete test DB folder -> run app -> create DB -> company setup -> admin signup -> QR -> login -> OTP -> dashboard.
- Navigation test: click all 23 sidebar keys and confirm no import error.
- Business test: create product, supplier, purchase invoice, stock update, customer, due payment, loyalty update.
- Performance test: app opens quickly; pages lazy-load; product table search does not freeze UI.
- Package test: install/run EXE on Windows 10/11 and create DB successfully in AppData.

13. Codex completion definition

Codex should finish only when all 41 target files exist, all existing files are either migrated or wrapped/archived, PyQt5 is removed from final runtime imports, all routes open, and the EXE can create a fresh DB per computer.